Why AI systems don't learn and what to do about it

Lessons on autonomous learning from cognitive science

Emmanuel Dupoux[1,2], **Yann LeCun**[3], **Jitendra Malik**[1,4]

[1]FAIR at META, [2]École des Hautes Études en Sciences Sociales, [3]NYU, [4]UC Berkeley

Presented by:

John Tan Chong Min

How do kids learn?

https://www.freepik.com/free-photos-vectors/playing-baby

- Self-directed (random) play
- Imitate a caregiver / older sibling
- Imagine ways of playing with the toy

Traditional versus Autonomous Machine Learning

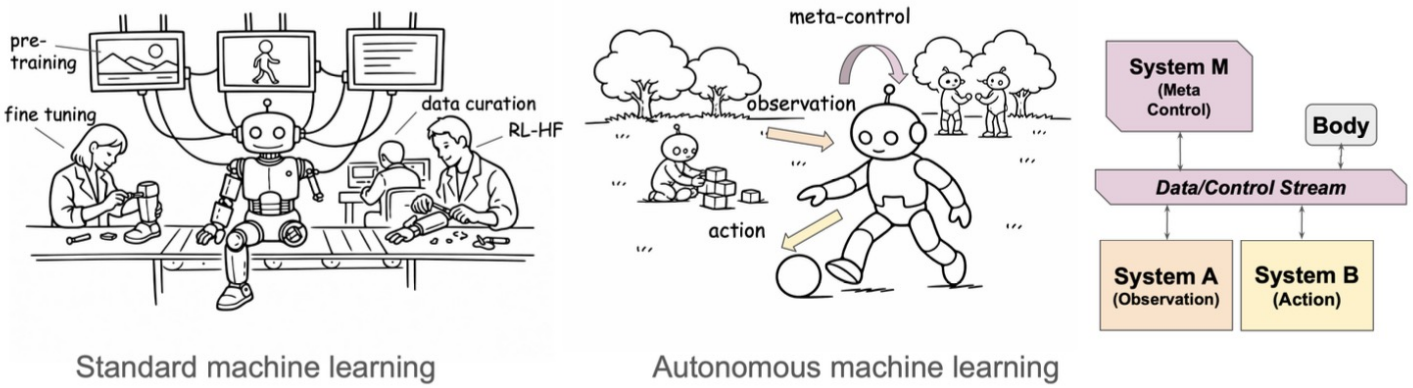
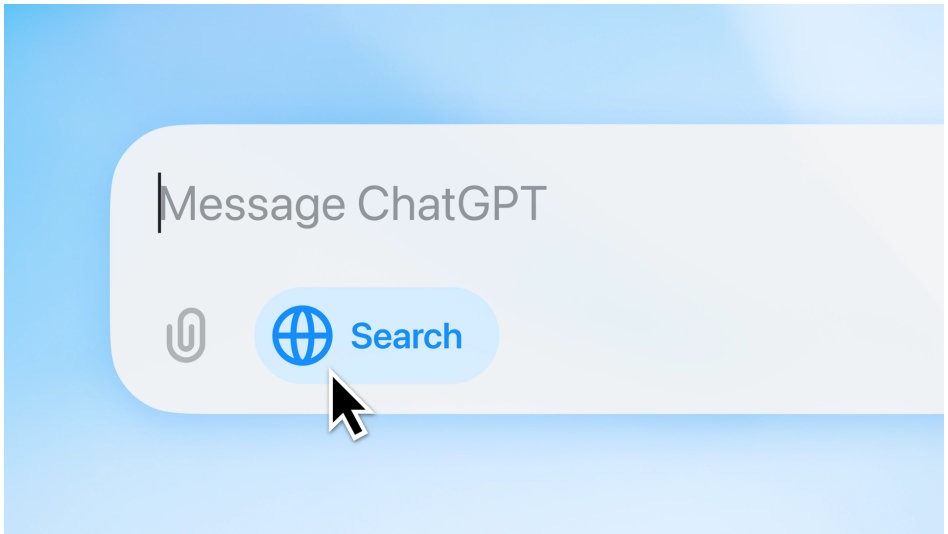


Figure 1 Standard machine learning (left). The machine does not learn by itself; it requires an assembly line of research engineers and data scientists collecting, formatting, and curating different kinds of data, each used to train successively different components of the model, each with specificly engineered loss and reward functions. The machine is then left with no ability to learn from its experience. **Autonomous machine learning (right).** The agent is learning directly in interaction with the world; the sources of data are generarated by the agent itself through different learning modes (learning by observation, by action, which can be extended to higher modes like learning by verbal interaction or self-play). Our proposed architecture include a meta controler enabling learning while operating in the real world. *(Drawings from ChatGPT).*

Large Language Models are frozen when training is completed



- Perhaps changing memory context / data store / imbuing web search can grant it new info at test time
- However, new info may not be incorporated well if too far out of training distribution

https://openai.com/index/introducing-chatgpt-search/

The real world has no labelled data – we learn from what we observe

LLMs do this observation part by doing self-supervised learning on next token

System A – Learning from Observation

Table 1 Sample observational learning problems (System A) from different sensory inputs, proposed mechanisms with corresponding terminology in Cognitive Science and AI fields and sample AI algorithms (non-exhaustive).

Input	Task	Proposed Learning Mechanisms		(sample) AI Algorithms
		CogSci terminology	AI terminology	
Speech	Learning phonetic categories	Distributional Learning / Perceptual Learning	Self-Supervised Learning / Acoustic Unit Discovery	CPC (van den Oord et al., 2018), HuBERT (Hsu et al., 2021)
Language	Syntax acquisition	Statistical Learning	Language Modeling	GPT (Radford et al., 2018), BERT (Devlin et al., 2019), GSLM (Lakhotia et al., 2021)
Images	Face recognition, Object categorization	Perceptual Learning	Self-Supervised Learning	SimCLR (Chen et al., 2020), MoCo (He et al., 2020), DINO (Caron et al., 2021), I-JEPA (Assran et al., 2023)
Video	Intuitive physics	Perceptual Learning	Predictive World Modeling	PredNet (Lotter et al., 2017), V-JEPA (Bardes et al., 2024)
Language + Vision	Learning words and sentence meaning	Cross-situational Learning	Multimodal Language Modeling	CLIP (Radford et al., 2021), Flamingo (Alayrac et al., 2022), Transfusion (Zhou et al., 2024)

System A Formulation
(Much like supervised / self-supervised learning)

$$\theta^\star = \arg\min_\theta \; \mathbb{E}_{x\sim\mathcal{D}} \, \mathcal{L}\big(f_\theta(x_{\text{in}}), x_{\text{tar}}\big)$$

Predict the x_target given x_input, which are from the real-world distribution D

Optimise parameters θ to make accurate predictions

* May or may not have actions involved (see later slides)

We also learn from our interactions with the world via our actions

Reinforcement Learning does this part, but not yet incorporated into a complete system with Self-Supervised learning

System B – Learning from Actions

Table 2 Popular paradigms addressing System B optimization problems, differing in how the world model M and the policy π are defined, whether they are learned from data in a training phase, and how they are used at inference time, with selected examples of applications in the modeling of animal behavior.

Paradigm	W. Model M (train., infer.)	Policy π (train., infer.)	When is optimization solved?	Example
Control Theory	fixed (no,no)	derived analytically (no, direct)	offline (design time)	Spinal reflexes, saccadic eye movement (Robinson, 1981)
Adaptive Control	adaptive param. (yes, no)	derived analytically (no, direct)	online (parameter estimation)	Motor adaptation (Shadmehr and Mussa-Ivaldi, 1994)
Model-Free RL	none	NN (yes, direct)	training time (compil-ation of experience into reactive policy)	{Habitual actions (Schultz et al., 1997)
Model-Based RL	NN (yes, no / unrolled)	NN (yes, direct / search)	both training and inference	Goal-directed behaviors, foraging strategies (Daw et al., 2011)
Planning	simulator / NN (no/yes, unrolled)	computed online (no,search)	inference time (thinking before acting)	Detour planning, mental simulation (Pfeiffer and Foster, 2013)

System B Formulation
(much like reinforcement learning)

$$\text{Maximize } J(\pi) = \mathbb{E}\left[\sum_{t=0}^{T} \gamma^t r(s_t, a_t)\right]$$

Where:

$s_{t+1} \sim M(.\|s_t, a_t)$	Dynamic model of the world
$a_t \sim \pi(.\|s_t)$	Agent's policy
$r(s, a)$	Reward function
$\gamma \in [0, 1)$	Discount factor

Maximise reward (can be goal-directed reward) by performing optimal sequence of actions in the world

** Note the model of the world can be stochastic (probabilistic), we still optimise expected reward (e.g. Poker)

System A and System B can help one another

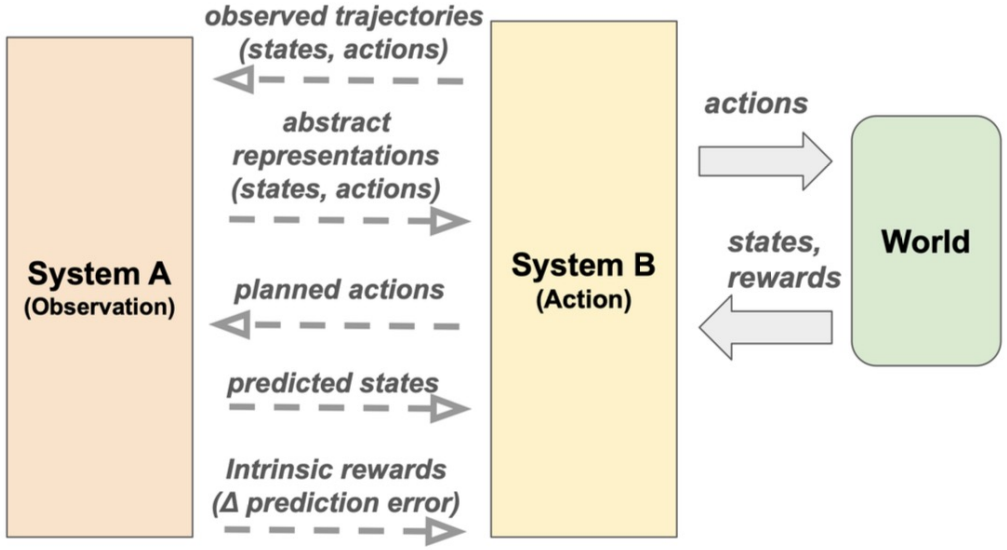


Figure 2 Summary of modes of interactions between Systems A and B. System A provides System B with predictions of future states conditioned on past states and actions, with hierarchical abstractions over possible actions, and a SSL loss that can be used for curiosity/exploration. System B through its action provides rich and task relevant input for System A to learn from.

System A and System B can help one another

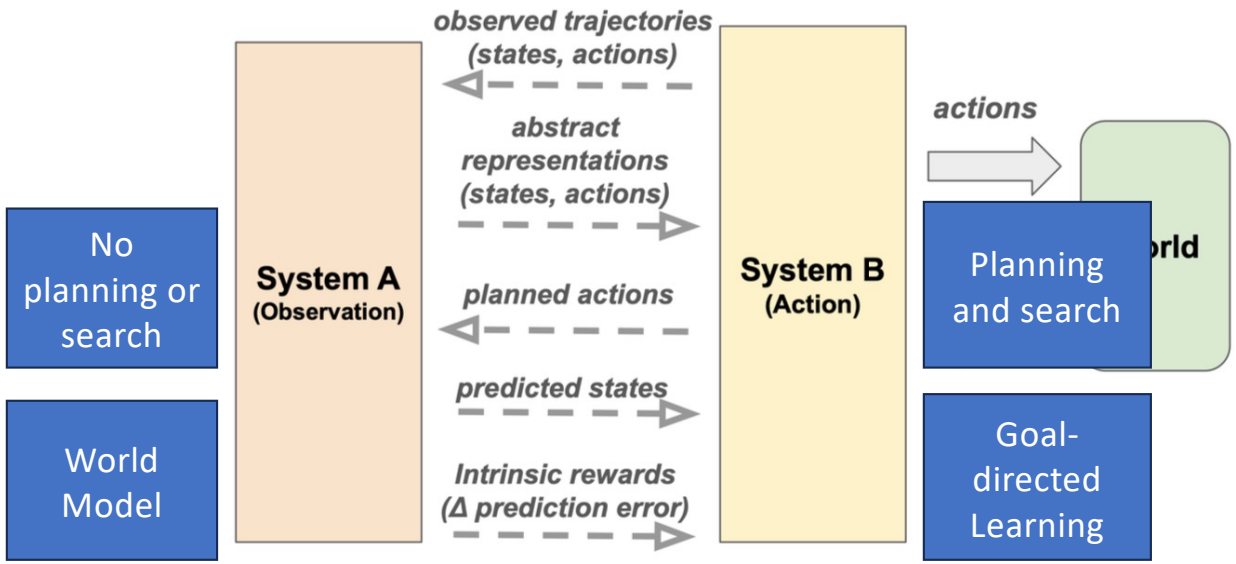


Figure 2 Summary of modes of interactions between Systems A and B. System A provides System B with predictions of future states conditioned on past states and actions, with hierarchical abstractions over possible actions, and a SSL loss that can be used for curiosity/exploration. System B through its action provides rich and task relevant input for System A to learn from.

System A and System B Interactions

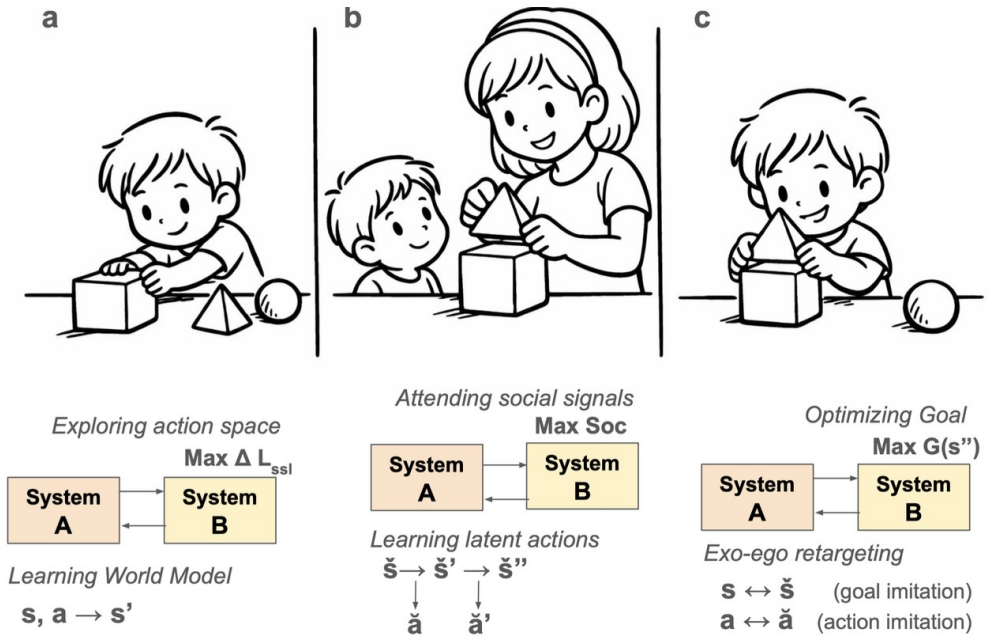
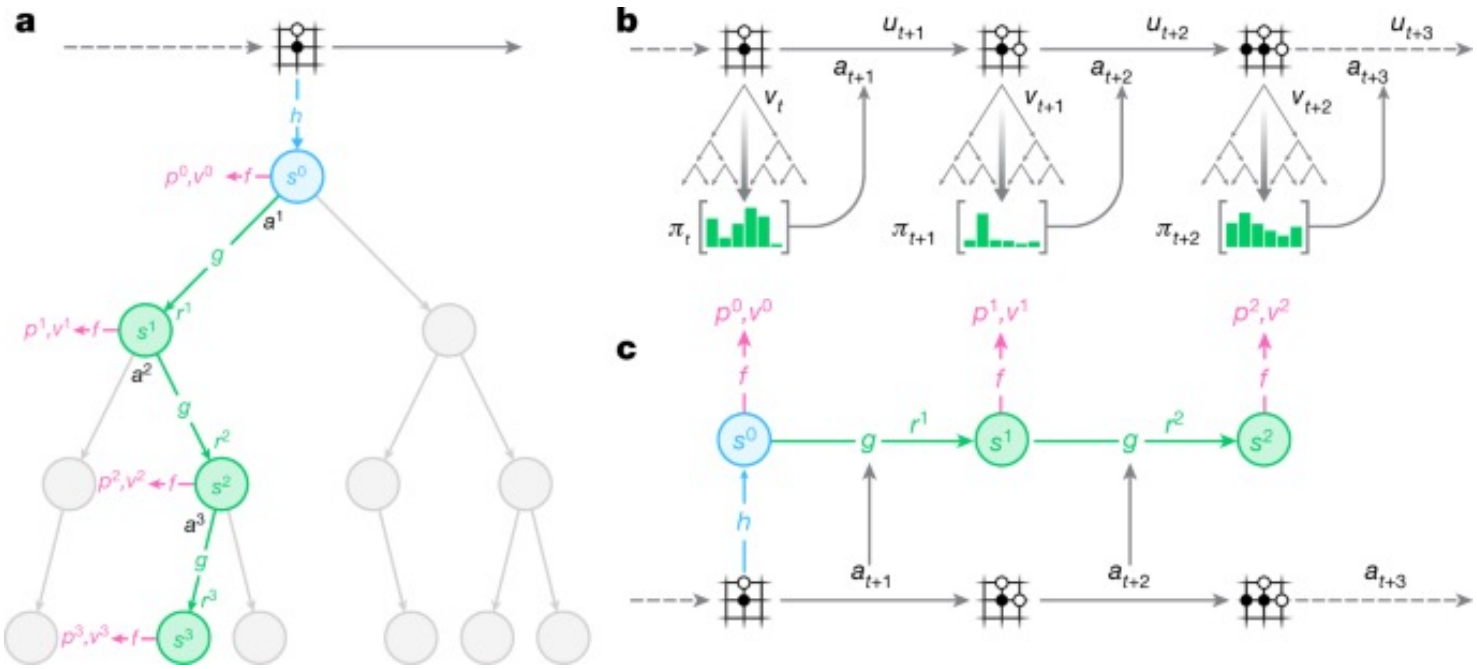


Figure 3 Interactions between learning modes for imitation learning. (a) **Self Play.** System B provides action, state trajectories to System A that learns a World Model, and provides a prediction-based intrinsic reward signal to system B. (b) **Social Observation**. System B directs the attention to peers that provide System A with complex trajectories from which it infers latent actions. (c) **Retargeted imitation**. System A learns to map exocentric actions and states to egocentric ones, helping system B to achieve goal-directed behavior. *(image from ChatGPT)*

MuZero: Example of System A + System B



Abstraction of state *s* by neural network *h*

Prediction of policy (*p*) and value networks (*v*) by neural network f

World model *g* gives the reward *r* and future state s when given state-action trajectories

All networks learned jointly

Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model. 2020. Google DeepMind.

How does System A and System B coordinate with one another

We need an orchestrator – in the paper they name it System M

System M – Choosing data streams for learning

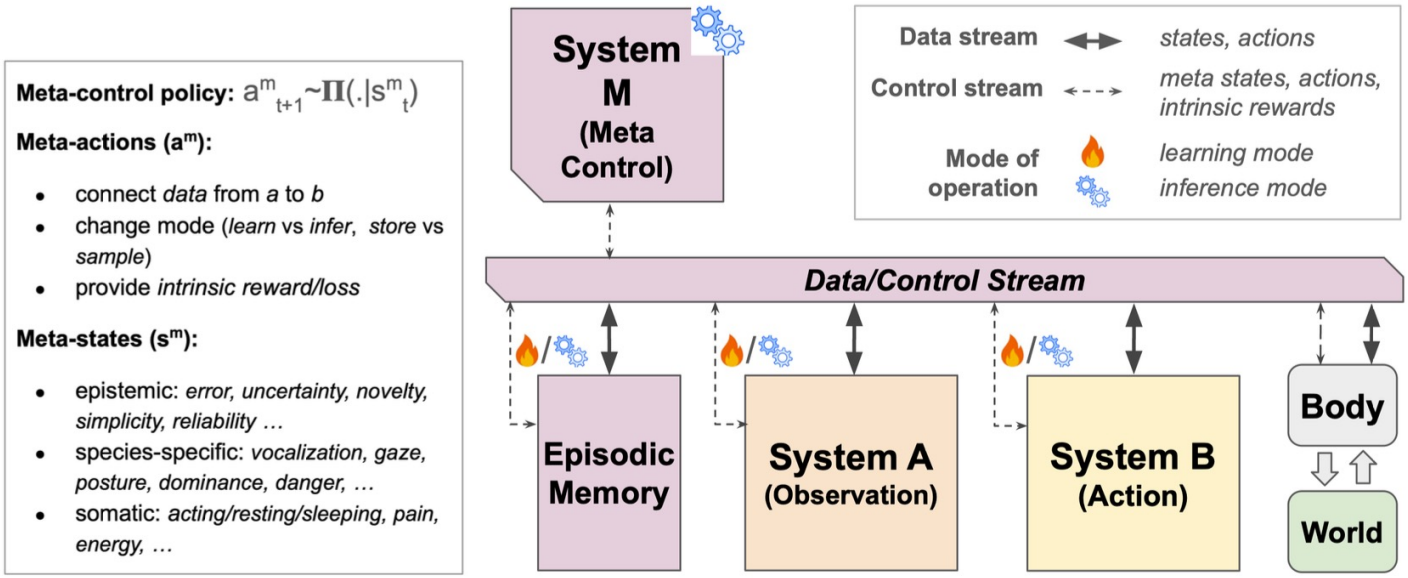


Figure 4 Blueprint of a cognitive architecture featuring System M as an autonomous orchestrator. System M acts as a central control plane that automates data routing and training recipes. High-bandwidth *data streams* (e.g., plain arrows) carry raw sensory inputs, motor commands, and latent representations between System A (perception/world modeling), System B (action/policy), and an episodic memory buffer. Low-bandwidth *control streams* (e.g., thin dashed arrows) carry telemetry: System M monitors internal meta-states (such as prediction errors or uncertainty) and outputs routing commands (meta-actions) to dynamically open or close data pathways, effectively assembling and disassembling learning and inference pipelines on the fly.

How can System A, B and M improve itself over generations?

We need a system architect to optimize the parameters?

Meta-loop configuring individual's parameters

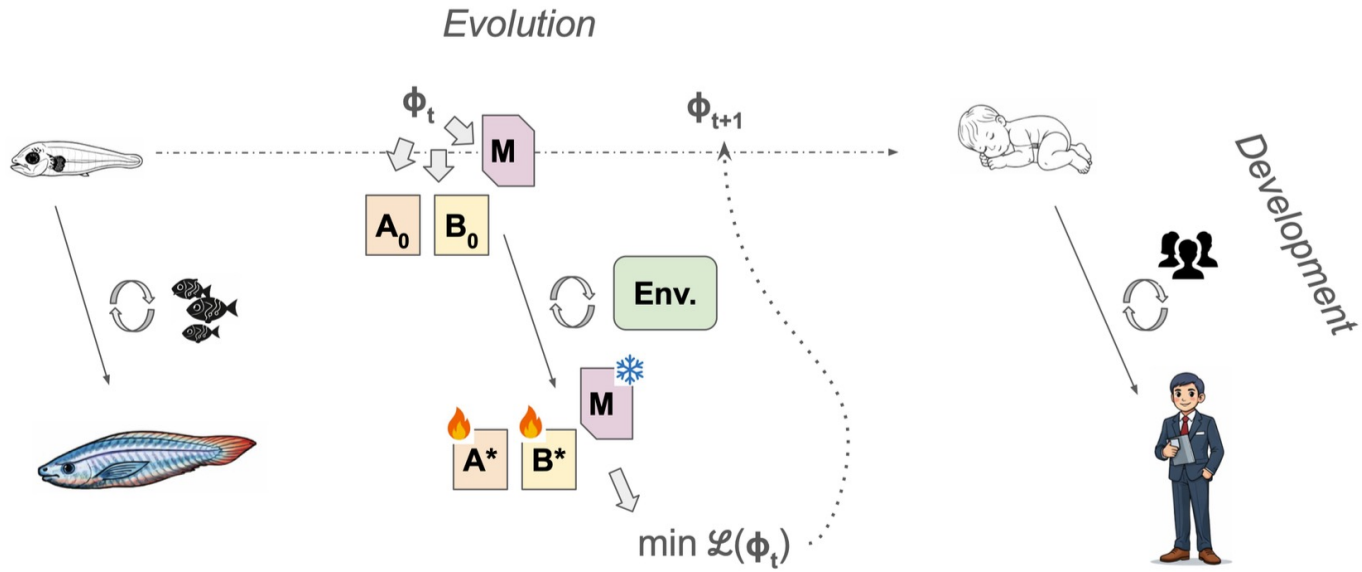


Figure 5 Evo/Devo framework for building autonomous learning agents. Learning takes place at two scales. In the developmental scale, the learner's architecture (A, B and M) is initialized from meta parameter ϕ. A and B update their parameters through interaction with the environment controlled by a fixed controler M. In the the evolutionary scale, *phi* is updated to optimize a fitness function $\mathcal{L}$ measured over the life cycle of the system. *(images from ChatGPT)*.

Meta-loop configuring individual's parameters

$$\phi_{t+1} = \arg\min_{\phi_t} \mathcal{L}(A_0 : A_K, B_0 : B_K)$$

subject to

$$A_0, B_0, M = \text{Init}(\phi_t)$$
$$A_{i+1}, B_{i+1} = \text{Update}(M, A_i, B_i, Env)$$

where

Init is the initialization procedure
Update is the inner loop update rule
Env is the interactive environment

System Architect Level Objective Function

- Maximise the fitness of individuals across lifes

<u>Food for thought:</u>
Can we just let natural selection do this job? Is there an objective function for natural selection or you just need enough diversity and growth? Must everything be a loss function?

Question to Ponder (1/2)

- Does System A need actions as well to model the world? How can we infer the actions of others if it is not our own trajectory?

- How can we learn so many objectives at once for System A, System B, System M? Is backpropagation still viable? Can there be other training methods to fulfil multiple objectives?

- MuZero / AlphaZero suffers a lot from bad initial configuration of the network due to early experience runs. How can we mitigate that in this System M as well?

Question to Ponder (2/2)

- How about multiple agents and group learning? How can we add group learning dynamics to this framework?

- Must everything be a loss function? Can natural selection and diversity already do the job to select the parameters

- Must everything be optimal? Will just satisficing solutions be good enough and we let the environment select the agents to be carried on to the next generation?